DATA MINING-II

13 May 2026 12:20

Frequent patterns are patterns (e.g., itemsets, subsequences, or substructures) that appear frequently in a data set. For example, a set of items, such as milk and bread, that appear frequently together in a transaction data set is a frequent itemset.
A subsequence, such as buying first a PC, then a digital camera, and then a memory card, if it occurs frequently in a shopping history database, is a (frequent) sequential pattern

it helps in data classification, clustering, and other data mining tasks

Basic Concepts:

Frequent pattern mining searches for recurring relationships in a given data set.
basic concepts of frequent pattern mining for the discovery of interesting associations and correlations between itemsets in transactional and relational databases.

Market Basket Analysis:

This process analyzes customer buying habits by finding associations between the different items that customers place in their "shopping baskets"
For instance, if customers are buying milk, how likely are they to also buy bread (and what kind of bread) on the same trip to the supermarket?

If we think of the universe as the set of items available at the store, then each item has a Boolean variable representing the presence or absence of that item. Each basket can then be represented by a Boolean vector of values assigned to these variables. The Boolean vectors can be analyzed for buying patterns that reflect items that are frequently associ ated or purchased together. These patterns can be represented in the form of association rules. For example, the information that customers who purchase computers also tend to buy antivirus software at the same time is represented in the following association rule:

computer ⇒ antivirus software [support = 2%,confidence = 60%].

Rule support and confidence are two measures of rule interestingness.
They respectively reflect the usefulness and certainty of discovered rules.
A support of 2% for Rule means that 2% of all the transactions under analysis show that computer and antivirus software are purchased together.
A confidence of 60% means that 60% of the customers who purchased a computer also bought the software.
Typically, association rules are considered interesting if they satisfy both a minimum support threshold and a minimum confidence threshold.
These thresholds can be a set by users or domain experts.

An association rule is an implication of the form $A \Rightarrow B, where A \subset I, B \subset I, A = \emptyset, B = \emptyset, and A \cap B = \phi$.

The rule $A \Rightarrow B$ holds in the transaction set D with support s, where s is the percentage of transactions in D that contain $A \cup B$ (i.e., the union of sets A and B say, or, both A and B). This is taken to be the probability, $P(A \cup B)$.1

The rule $A \Rightarrow B$ has confidence c in the transaction set D, where c is the percentage of transactions in D containing A that also contain B. This is taken to be the conditional probability, $P(B|A)$. That is,

$support(A \Rightarrow B) = P(A \cup B)$
$confidence(A \Rightarrow B) = P(B|A)$.

Rules that satisfy both a minimum support threshold (min sup) and a minimum confidence threshold (min conf) are called strong.

Apriori Algorithm:
==========
Apriori is a seminal algorithm proposed by R. Agrawal and R. Srikant in 1994 for min ing frequent itemsets for Boolean association rules
The name of the algorithm is based on the fact that the algorithm uses prior knowledge of frequent itemset properties,
Apriori employs an iterative approach known as a level-wise search, where k-itemsets are used to explore (k +1)-itemsets.
First, the set of frequent 1-itemsets is found by scanning the database to accumulate the count for each item, and collecting those items that satisfy minimum support. The resulting set is denoted by L1. Next, L1 is used to find L2, the set of frequent 2-itemsets, which is used to find L3, and so on, until no more frequent k-itemsets can be found.

To improve the efficiency of the level-wise generation of frequent itemsets, an important property called the Apriori property is used to reduce the search space.

Apriori Property:

Apriori property: All nonempty subsets of a frequent itemset must also be frequent.
By definition, if an item set I does not satisfy the minimum support threshold, min sup, then I is not frequent, that is, $P(I) < min\ sup$.

If an item A is added to the itemset I, then the resulting itemset (i.e., $I \cup A$) cannot occur more frequently than I. Therefore, $I \cup A$ is not frequent either, that is, $P(I \cup A) < min\ sup$.

This will use both Join and prune set

Example:

There are nine transactions in this database, that is, |D| = 9. Weuse Figure 6.2 to illustrate the Apriori algorithm for finding frequent itemsets in D.

1. In the first iteration of the algorithm, each item is a member of the set of candidate 1-itemsets, C1. The algorithm simply scans all of the transactions to count the number of occurrences of each item.

2. Suppose that the minimum support count required is 2, that is, min sup = 2.
(Here, weare referring to absolute support because we are using a support count. The corre sponding relative support is 2/9 = 22%.) The set of frequent 1-itemsets, L1, can then be determined. It consists of the candidate 1-itemsets satisfying minimum support.

3. To discover the set of frequent 2-itemsets, L2, the algorithm uses the join L1 L1 to generate a candidate set of 2-itemsets, C2.7 C2 consists of |L1| 2 2-itemsets. Note that no candidates are removed from C2 during the prune step because each subset of the candidates is also frequent

4. Next, the transactions in D are scanned and the support count of each candidate itemset in C2 is accumulated,

5. The set of frequent 2-itemsets, L2, is then determined, consisting of those candidate 2-itemsets in C2 having minimum support.

6.The generation of the set of the candidate 3-itemsets, C3, is detailed in Figure 6.3. From the join step, we first get C3 = L2 L2 = {{I1, I2, I3}, {I1, I2, I5}, {I1, I3, I5}, {I2, I3, I4}, {I2, I3, I5}, {I2, I4, I5}}. Based on the Apriori property that all subsets of a frequent itemset must also be frequent,

(a) Join: C3 = L2 L2 ={{I1,I2}, {I1, I3}, {I1, I5}, {I2, I3}, {I2, I4}, {I2, I5}} {{I1, I2}, {I1, I3}, {I1, I5}, {I2, I3}, {I2, I4}, {I2, I5}} ={{I1, I2, I3}, {I1, I2, I5}, {I1, I3, I5}, {I2, I3, I4}, {I2, I3, I5}, {I2, I4, I5}}.

(**b**) Prune using the Apriori property: All nonempty subsets of a frequent itemset must also be frequent. Do any of the candidates have a subset that is not frequent?

- The 2-item subsets of {I1, I2, I3} are {I1, I2}, {I1, I3}, and {I2, I3}. All 2-item subsets of {I1, I2, I3} are members of L_2. Therefore, keep {I1, I2, I3} in C_3.
- The 2-item subsets of {I1, I2, I5} are {I1, I2}, {I1, I5}, and {I2, I5}. All 2-item subsets of {I1, I2, I5} are members of L_2. Therefore, keep {I1, I2, I5} in C_3.
- The 2-item subsets of {I1, I3, I5} are {I1, I3}, {I1, I5}, and {I3, I5}. {I3, I5} is not a member of L_2, and so it is not frequent. Therefore, remove {I1, I3, I5} from C_3.
- The 2-item subsets of {I2, I3, I4} are {I2, I3}, {I2, I4}, and {I3, I4}. {I3, I4} is not a member of L_2, and so it is not frequent. Therefore, remove {I2, I3, I4} from C_3.
- The 2-item subsets of {I2, I3, I5} are {I2, I3}, {I2, I5}, and {I3, I5}. {I3, I5} is not a member of L_2, and so it is not frequent. Therefore, remove {I2, I3, I5} from C_3.
- The 2-item subsets of {I2, I4, I5} are {I2, I4}, {I2, I5}, and {I4, I5}. {I4, I5} is not a member of L_2, and so it is not frequent. Therefore, remove {I2, I4, I5} from C_3.

(**c**) Therefore, $C_3 = \{\{I1, I2, I3\}, \{I1, I2, I5\}\}$ after pruning.

Screen clipping taken: 23-05-2026 13:47

7. The transactions in D are scanned to determine L3, consisting of those candidate 3-itemsets in C3 having minimum support

8. The algorithm uses L3 L3 to generate a candidate set of 4-itemsets, C4. Although the join results in {{I1, I2, I3, I5}}, itemset {I1, I2, I3, I5} is pruned because its subset {I2, I3, I5} is not frequent. Thus, C4 = ϕ, and the algorithm terminates, having found all of the frequent itemsets.

Table 6.1 Transactional Data for an *AllElectronics* Branch

TID	*List of item_IDs*
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

Screen clipping taken: 23-05-2026 13:46

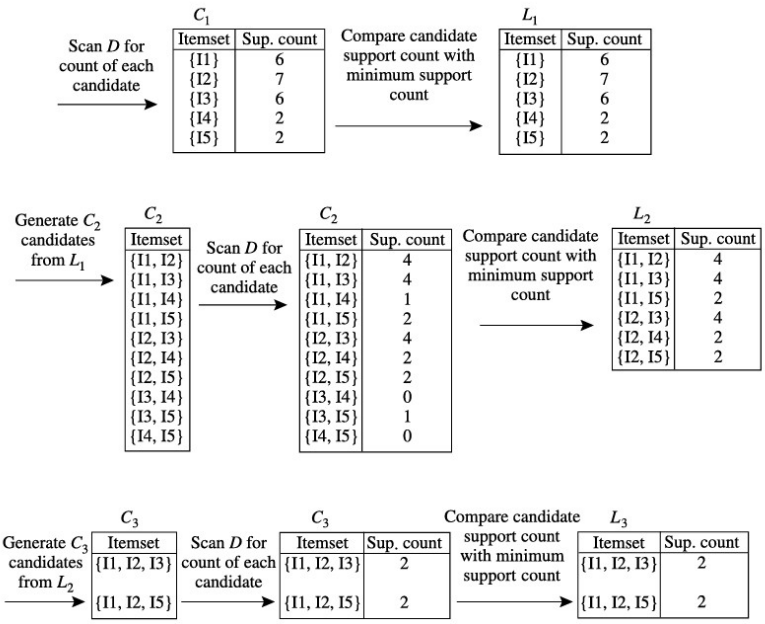

Figure 6.2 Generation of the candidate itemsets and frequent itemsets, where the minimum support count is 2.

Generating Association Rules from Frequent Itemsets:

Once the frequent itemsets from transactions in a database D have been found, it is straightforward to generate strong association rules from them (where strong association rules satisfy both minimum support and minimum confidence). This can be done using Eq. (6.4) for confidence, which we show again here for completeness:

$$confidence(A \Rightarrow B) = P(B|A) = \frac{support\_count(A \cup B)}{support\_count(A)}.$$

Screen clipping taken: 23-05-2026 13:59

where support count(AUB) is the number of transactions containing the itemsets AUB, and support count(A) is the number of transactions containing the itemset A.

Example 6.4 **Generating association rules.** Let's try an example based on the transactional data for *AllElectronics* shown before in Table 6.1. The data contain frequent itemset $X = \{I1, I2, I5\}$. What are the association rules that can be generated from X? The nonempty subsets of X are {I1, I2}, {I1, I5}, {I2, I5}, {I1}, {I2}, and {I5}. The resulting association rules are as shown below, each listed with its confidence:

$\{I1, I2\} \Rightarrow I5$, $confidence = 2/4 = 50\%$
$\{I1, I5\} \Rightarrow I2$, $confidence = 2/2 = 100\%$
$\{I2, I5\} \Rightarrow I1$, $confidence = 2/2 = 100\%$
$I1 \Rightarrow \{I2, I5\}$, $confidence = 2/6 = 33\%$
$I2 \Rightarrow \{I1, I5\}$, $confidence = 2/7 = 29\%$
$I5 \Rightarrow \{I1, I2\}$, $confidence = 2/2 = 100\%$

If the minimum confidence threshold is, say, 70%, then only the second, third, and last rules are output, because these are the only ones generated that are strong. Note that, unlike conventional classification rules, association rules can contain more than one conjunct in the right side of the rule. ■

Disadvantages of Apriori algorithm:

- *It may still need to generate a huge number of candidate sets.* For example, if there are 10^4 frequent 1-itemsets, the Apriori algorithm will need to generate more than 10^7 candidate 2-itemsets.
- *It may need to repeatedly scan the whole database and check a large set of candidates by pattern matching.* It is costly to go over each transaction in the database to determine the support of the candidate itemsets.

==========

A Pattern-Growth Approach for Mining Frequent Itemsets:
---

frequent pattern growth, or simply FP-growth, which adopts a divide-and-conquer strategy as follows.
First, it compresses the database representing frequent items into a frequent pattern tree, or FP-tree, which retains the itemset association information.
It then divides the compressed database into a set of conditional databases
each associated with one frequent item or "pattern fragment," and mines each database separately.

FP-growth (finding frequent itemsets without candidate generation). :
---

1. The first scan of the database is the same as Apriori, which derives the set of frequent items (1-itemsets) and their support counts (frequencies).
2. Let the minimum support count be 2. The set of frequent items is sorted in the order of descending support count.
3. This resulting set or list is denoted by L. Thus, we have L ={{I2: 7}, {I1: 6}, {I3: 6}, {I4: 2}, {I5: 2}}.
4. An FP-tree is then constructed as follows. First, create the root of the tree, labeled with "null." Scan database D a second time.
5. The items in each transaction are processed in L order (i.e., sorted according to descending support count), and a branch is created for each transaction.

For example,
the scan of the first transaction, "T100: I1, I2, I5," which contains three items (I2, I1, I5 in L order), leads to the construction of the first branch of the tree with three nodes, I2: 1 , I1: 1 , and I5: 1 , where I2 is linked as a child to the root, I1 is linked to I2, and I5 is linked to I1.

The second transaction, T200, contains the items I2 and I4 in L order, which would result in a branch where I2 is linked to the root and I4 is linked to I2.
However, this branch would share a common prefix, I2, with the existing path for T100. Therefore, we instead increment the count of the I2 node by 1, and create a new node, I4: 1 , which is linked as a child to I2: 2 .

The FP-tree is mined as follows. Start from each frequent length-1 pattern
construct its conditional pattern base then construct its (conditional) FP-tree, and perform mining recursively on the tree.
The pattern growth is achieved by the concatenation of the suffix pattern with the frequent patterns generated from a conditional FP-tree.

The FP-growth method transforms the problem of finding long frequent patterns into searching for shorter ones in much smaller conditional databases recursively and then concatenating the suffix.

It uses the least frequent items as a suffix, offering good selectivity. The method substantially reduces the search costs.

When the database is large, it is sometimes unrealistic to construct a main memory based FP-tree

An interesting alternative is to first partition the database into a set of projected databases, and then construct an FP-tree and mine it in each projected database.

This process can be recursively applied to any projected database if its FP-tree still cannot fit in main memory

==========\\\

WhichPatterns Are Interesting?—Pattern Evaluation Methods

Most association rule mining algorithms employ a support–confidence framework.
Although minimum support and confidence thresholds help weed out or exclude the exploration of a good number of uninteresting rules,
many of the rules generated are still not interesting to the users.
Unfortunately, this is especially true when mining at low support thresholds or mining for long patterns.
This has been a major bottleneck for successful application of association rule mining.

1. support–confidence frame work
2. correlation analysis

1. Strong Rules Are Not Necessarily Interesting
2. From Association Analysis to Correlation Analysis
3. A Comparison of Pattern Evaluation Measures

1. Strong Rules Are Not Necessarily Interesting:

"How can we tell which strong association rules are really interesting?" Let's examine the following example.
A misleading "strong" association rule.

1. From Association Analysis to Correlation Analysis:

As we have seen so far, the support and confidence measures are insufficient at filtering out uninteresting association rules.
To tackle this weakness, a correlation measure can be used to augment the support–confidence framework for association rules.
This leads to correlation rules of the form A⇒B[support, confidence, correlation].

That is, a correlation rule is measured not only by its support and confidence but also by the correlation between itemsets A and B.
There are many different correlation measures from which to choose.

Lift is a simple correlation measure that is given as follows.
The occurrence of itemset A is independent of the occurrence of itemset B if P(A∪B) = P(A)P(B); otherwise, itemsets A and B are dependent and correlated as events.

$$lift(A, B) = \frac{P(A \cup B)}{P(A)P(B)}.$$

If the resulting value of Eq. (6.8) is less than 1, then the occurrence of A is negatively correlated with the occurrence of B, meaning that the occurrence of one likely leads to the absence of the other one.

If the resulting value is greater than 1, then A and B are positively correlated, meaning that the occurrence of one implies the occurrence of the other.

If the resulting value is equal to 1, then A and B are independent and there is no correlation between them.

The second correlation measure that we study is the χ2 measure,
To compute the χ2 value, we take the squared difference between the observed and expected value for a slot (A and B pair) in the contingency table, divided by the expected value.

Correlation analysis using χ2.

To compute the correlation using χ2 analysis for nominal data ,we need the observed value and expected value (displayed in parenthesis)for each slot of the contingency table, as shown inTable6.7.From the table, we can compute the χ2 value as follows:

$$\chi^2 = \Sigma \frac{(observed - expected)^2}{expected} = \frac{(4000 - 4500)^2}{4500} + \frac{(3500 - 3000)^2}{3000}$$
$$+ \frac{(2000 - 1500)^2}{1500} + \frac{(500 - 1000)^2}{1000} = 555.6.$$

Screen clipping taken: 25-05-2026 08:05

Table 6.7 Table 6.6 Contingency Table, Now with the Expected Values

	game	$\overline{game}$	Σ_{row}
video	4000 (4500)	3500 (3000)	7500
$\overline{video}$	2000 (1500)	500 (1000)	2500
Σ_{col}	6000	4000	10,000

Screen clipping taken: 25-05-2026 08:05

A Comparison of Pattern Evaluation Measures:
==========
Instead of using the simple support–confidence frame work to evaluate frequent patterns, other measures, such as lift and χ2, often disclose more intrinsic pattern relationships. How effective are these measures? Should we also consider other alternatives?

Recently, several other pattern evaluation measures have attracted interest.

all confidence,
max confidence,
Kulczynski, and
cosine.

Given two itemsets, A and B, the **all_confidence** measure of A and B is defined as

$$all\_conf(A, B) = \frac{sup(A \cup B)}{max\{sup(A), sup(B)\}} = min\{P(A|B), P(B|A)\}, \quad (6.9)$$

where $max\{sup(A), sup(B)\}$ is the maximum support of the itemsets A and B. Thus, $all\_conf(A, B)$ is also the minimum confidence of the two association rules related to A and B, namely, "$A \Rightarrow B$" and "$B \Rightarrow A$."

Screen clipping taken: 25-05-2026 08:38

Given two itemsets, A and B, the **max_confidence** measure of A and B is defined as

$$max\_conf(A, B) = max\{P(A \mid B), P(B \mid A)\}. \quad (6.10)$$

The max_conf measure is the maximum confidence of the two association rules, "$A \Rightarrow B$" and "$B \Rightarrow A$."

Screen clipping taken: 25-05-2026 08:38

Given two itemsets, A and B, the **Kulczynski** measure of A and B (abbreviated as **Kulc**) is defined as

$$Kulc(A, B) = \frac{1}{2}(P(A|B) + P(B|A)). \quad (6.11)$$

It was proposed in 1927 by Polish mathematician S. Kulczynski. It can be viewed as an average of two confidence measures. That is, it is the average of two conditional probabilities: the probability of itemset B given itemset A, and the probability of itemset A given itemset B.

Screen clipping taken: 25-05-2026 08:39

Finally, given two itemsets, A and B, the **cosine** measure of A and B is defined as

$$cosine(A, B) = \frac{P(A \cup B)}{\sqrt{P(A) \times P(B)}} = \frac{sup(A \cup B)}{\sqrt{sup(A) \times sup(B)}}$$
$$= \sqrt{P(A|B) \times P(B|A)}. \quad (6.12)$$

The *cosine* measure can be viewed as a *harmonized lift* measure: The two formulae are similar except that for cosine, the *square root* is taken on the product of the probabilities of A and B. This is an important difference, however, because by taking the square root, the cosine value is only influenced by the supports of A, B, and $A \cup B$, and not by the total number of transactions.

Screen clipping taken: 25-05-2026 08:39

"*Why are* lift *and* χ^2 *so poor at distinguishing pattern association relationships in the previous transactional data sets?*" To answer this, we have to consider the *null-transactions*. A **null-transaction** is a transaction that does not contain any of the itemsets being examined. In our example, $\overline{mc}$ represents the number of null-transactions.

Screen clipping taken: 25-05-2026 08:41

"*Among the* all_confidence, max_confidence, Kulczynski, *and* cosine *measures, which is best at indicating interesting pattern relationships*?"

To answer this question, we introduce the **imbalance ratio** (**IR**), which assesses the imbalance of two itemsets, A and B, in rule implications. It is defined as

$$IR(A, B) = \frac{|sup(A) - sup(B)|}{sup(A) + sup(B) - sup(A \cup B)}, \quad (6.13)$$

Screen clipping taken: 25-05-2026 08:42

where the numerator is the absolute value of the difference between the support of the itemsets A and B, and the denominator is the number of transactions containing A or B. If the two directional implications between A and B are the same, then $IR(A, B)$ will be zero. Otherwise, the larger the difference between the two, the larger the imbalance ratio. This ratio is independent of the number of null-transactions and independent of the total number of transactions.

Screen clipping taken: 25-05-2026 08:43

the use of only support and confidence measures to mine associations may generate a large number of rules, many of which can be uninteresting to users.

Instead, we can augment the support–confidence framework with a pattern interestingness measure, which helps focus the mining toward rules with strong pattern relationships.

Unfortunately, most of them do not have the null-invariance property. Because large data sets typically have many null-transactions, it is important to consider the null invariance property when selecting appropriate interestingness measures for pattern evaluation.

Among the four null-invariant measures studied here, namely all confidence, max confidence, Kulc, and cosine, we recommend using Kulc in conjunction with the imbalance ratio.s